



VRIJE
UNIVERSITEIT
BRUSSEL



Solar System Visualization and Simulation

Advanced Programming Concepts Project Report

Ming Yang Lei Xiong

May 20, 2026

1 Introduction

This report presents our Advanced Programming Concepts project: a Python visualization and simulation of the Solar System. The project applies object-oriented programming to model the Sun, planets, moons, Pluto, and a rocket-related extension as related objects with shared and specialized behavior.

The simulator is implemented with `pygame`. It displays the Solar System in two dimensions and uses a simplified orbital model to show how celestial bodies move over time. This report describes the project goals, object-oriented design, simulation model, user interface, results, limitations, and team contributions.

2 Project Objectives and Design Choices

We interpreted the assignment as more than a drawing exercise. A minimal program could place circles on the screen and rotate them with fixed angles, but that would not make good use of object-oriented design and would not show how physical quantities affect motion. Therefore, the implementation was designed around three principles.

First, the Solar System should be represented as a set of objects with state. Each body has a name, description, mass, radius, position, velocity, color, and recent trail. These values belong to the object itself instead of being stored as unrelated global variables.

Second, shared behavior should be placed in base classes. Drawing a body, storing its position, and updating its trail are common operations. They should not be copied separately into every planet or moon class. This keeps the design closer to the real relationship between the objects.

Third, the simulation should make design tradeoffs explicit. Some physical details are simplified, but the simplifications are intentional. The Sun is kept fixed to make the view stable, distances and masses are scaled for visibility, and moons use local orbits so they remain readable in the compressed display.

3 System Design

The code is divided into small modules with separate responsibilities. The `core` package contains the object model, the `engine` package contains the gravity calculation, and the `simulation` package connects the world model to the `pygame` event loop. The optional `easter_egg` package contains the separate solar-return visualization.

Table 1: Main parts of the implementation

File or package	Role in the project
<code>main.py</code>	Starts the application and creates the simulation controller
<code>config.py</code>	Stores body data, colors, screen size, and physics constants
<code>core/</code>	Defines stars, planets, moons, dwarf planets, and rockets
<code>engine/physics_engine.py</code>	Computes gravitational acceleration and movement
<code>simulation/solar_system.py</code>	Builds the Sun, planets, moons, and Pluto
<code>simulation/simulation.py</code>	Handles input, drawing, frame timing, and runtime state
<code>easter_egg/</code>	Provides the optional solar-return demonstration

3.1 Class Model

The class hierarchy follows the idea that all visible bodies are objects, but not all objects behave in the same way:

```
GameObject
  -> CelestialBody
      -> Star
      -> Planet
          -> DwarfPlanet
      -> Moon
      -> Rocket
```

`GameObject` is the smallest common abstraction. It stores identity and position. `CelestialBody` adds the physical and visual properties needed by a moving astronomical object: mass, radius, color, velocity, update logic, and trail drawing. This base class prevents repeated code in the subclasses.

`Star`, `Planet`, `Moon`, `DwarfPlanet`, and `Rocket` then specialize this common model. The Sun is represented by `Star`; it contributes gravity but remains fixed on the screen. Planets move under the global physics engine and may own moons. A moon stores a reference to its parent planet and updates its position relative to that parent. Pluto is implemented as a `DwarfPlanet`, inheriting most planet behavior while adding a classification label and a different visual trail.

This design was chosen because it keeps the program extensible. Adding a new planet mostly means adding data in `config.py`. Adding a new type of body, such as a comet or asteroid, could be done by deriving another class from `CelestialBody` or `Planet`, depending on the required behavior.

3.2 World Construction

The `SolarSystem` class creates the live objects from the configuration data. The eight planets are created from a list of dictionaries, which makes the setup concise and avoids writing eight separate blocks of nearly identical code. Earth, Jupiter, and their moons are then connected through parent-child relationships.

The same design also makes multiple instantiation natural rather than forced: `Planet` is used repeatedly for Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, and Neptune. The program therefore demonstrates that a class is a reusable model for many concrete objects, not just a container for one item.

4 Simulation Model

The movement of the main bodies is controlled by `PhysicsEngine`. At each frame, the engine computes the gravitational acceleration caused by every registered pair of bodies. It then updates all velocities and positions using the same snapshot of accelerations. This avoids a common simulation problem where the result depends on the order in which objects happen to be updated.

The model is based on Newtonian gravity. For two bodies with masses m_1 and m_2 at distance r , the force magnitude is

$$F = G \frac{m_1 m_2}{r^2}. \quad (1)$$

The engine uses the acceleration form of the same relation:

$$a_1 = G \frac{m_2}{r^2}, \quad a_2 = G \frac{m_1}{r^2}. \quad (2)$$

The direction of the acceleration is computed from the vector between the two bodies. A small softening value is added to the squared distance, so that the program does not produce an infinite acceleration if two objects are extremely close.

Positions are advanced with a simple symplectic Euler update:

$$v_{t+\Delta t} = v_t + a\Delta t, \quad (3)$$

$$x_{t+\Delta t} = x_t + v_{t+\Delta t}\Delta t. \quad (4)$$

This method is not as accurate as advanced numerical integration methods, but it is easy to explain, stable enough for the scaled simulation, and suitable for a real-time visual project.

4.1 Initial Conditions

Each planet starts at a chosen orbital radius from the Sun. Its initial tangential velocity is derived from the circular orbit equation

$$v = \sqrt{\frac{GM}{r}}, \quad (5)$$

where M is the mass of the central body and r is the orbital radius. The configuration also includes speed factors. These are visual tuning values that help keep orbits stable and observable at the selected scale.

The project uses scaled units. Distances are screen-space distances, masses are relative simulation masses, and the gravitational constant is chosen for visible movement. This scaling is necessary because real Solar System values span many orders of magnitude and cannot be displayed directly in a small window.

4.2 Moon Motion

Moons are handled differently from planets. They are updated in a local frame around their parent planet instead of being registered in the global gravity engine. At first this may look less physical, but it is a practical choice for this visualization. With the compressed distances used on screen, global Sun gravity would dominate and make the moons difficult to keep near their planets.

For each moon, the angular velocity is computed from its orbital speed and orbital radius:

$$\omega = \frac{v}{r}. \quad (6)$$

The moon's position is then recomputed from the parent position:

$$x = x_{parent} + r \cos(\theta), \quad y = y_{parent} + r \sin(\theta). \quad (7)$$

This gives a stable visual result while preserving the idea that moons depend on their parent planets.

5 Interface and Runtime Behavior

The program runs as an interactive `pygame` application. The `Simulation` class initializes the window, processes events, updates the Solar System, draws the frame, and maintains the configured frame rate.

Table 2: User controls	
Input	Effect
+ or keypad +	Increase simulation speed
- or keypad -	Decrease simulation speed
Mouse wheel	Zoom in or out
R	Open the solar-return demonstration
Esc or Q	Quit the program

The visual layer shows the bodies as colored disks with labels and orbit trails. The trails are useful because the user can see the recent motion of each body instead of only its current position. Trail length is limited so that memory usage remains bounded during long runs.

Zoom is implemented as a camera transformation during drawing. The physical coordinates are restored after each object is drawn, so user zoom does not change the underlying simulation state. This separation between model coordinates and screen coordinates makes the implementation easier to reason about.

The heads-up display is intentionally simple. It shows the current speed, current zoom, the available controls, and the number of tracked non-star bodies. The display is meant to support the simulation without covering the main visual content.

6 Additional Demonstration

The project includes an optional Qian Xuesen solar-return demonstration. This part is launched from the main window but runs in a separate process and window. It shows a simplified mission in which a rocket leaves Earth, follows a heliocentric ellipse, and returns after a set number of Earth years.

This additional feature was included for two reasons. First, it gives a more mission-oriented use of orbital mechanics than the main Solar System view. Second, it shows how the project structure can be extended without making the main simulation loop too complex. The main Solar System remains responsible for planetary visualization, while the mission demonstration has its own model and runtime window.

7 Results, Discussion, and Conclusion

The final program produces a working real-time visualization of the Solar System. The user can observe the planets moving around the Sun, see the Moon orbiting Earth, and see Io and Europa orbiting Jupiter. Pluto is included as a dwarf planet with a separate label, which makes its classification visible in the interface. The speed and zoom controls make the simulation easier to inspect, and the orbit trails help show motion over time instead of only showing the current position of each body.

From a programming point of view, the main result is the object structure behind the visualization. The bodies are not only drawn on screen; they are represented as objects with their own data and behavior. The gravity engine works on these objects, the renderer asks them to draw themselves, and the simulation controller coordinates updates and input. This separation makes the code easier to modify than a single-file procedural animation, and it keeps the connection between the real-world system and the code structure visible.

The project also shows the limits of this design. It is not an accurate astronomical ephemeris. The distances, radii, masses, and gravitational constant are scaled for visual clarity. The Sun is fixed, and moons use local circular orbits. These choices reduce scientific accuracy, but they make the simulation readable and keep the focus on object-oriented modelling. In that sense, the simplifications are part of the design: the project prioritizes a clear and extensible classroom simulation over physical precision.

A natural next step would be to add object selection. Clicking a planet could open an information panel showing its mass, radius, velocity, and description. Another improvement would be to add more realistic elliptical orbits, asteroid belts, comets, and direct rocket launches in the main window. These extensions would fit the current structure because the project already separates body objects, physics, and rendering.

Overall, the project meets its intended goal. It uses Python and **pygame** to build an interactive Solar System simulator whose structure is based on inheritance, composition, reusable classes, and multiple object instances. The final application is intentionally simplified, but it provides a coherent implementation of the project topic and a clear demonstration of the object-oriented concepts emphasized in the course.

8 Team Contribution

The work was divided across modelling, simulation logic, interface behavior, and documentation. The contribution summary below records the main areas of responsibility.

Table 3: Contribution summary

Member	Main contribution areas
Ming Yang	Core class model, celestial body data, object relationships, and code documentation.
Lei Xiong	Physics engine, simulation loop, design checks, and report preparation.